CS 182 - HW 5 - Jie Wen

1. Convolutional Networks

(a) Two reasons to prefer conv layers for images

1. **Far fewer parameters via weight sharing** → The same small kernel slides across the whole image, so a separate weight is not learned for every input pixel like a fully-connected layer would. This cuts memory/compute and reduces overfitting.
2. **Right inductive bias: locality + (approx.) translation equivariance** → Features detected in one location are detected elsewhere by the same kernel, so small shifts in the image produce correspondingly shifted feature maps.
   (In modern DL code, "conv" layers usually implement cross-correlation—no kernel flip—matching the given convention.)

(b) Find the length-3 filter

Let input $s = [1, 4, 0, -2, 3]$, filter $f = [a, b, c]$, stride 1, no padding, and (per the given convention) no flip. The output is $[-2, 2, 11]$, so

$$\begin{aligned} a + 4b &= -2 && (y_0) \\ 4a - 2c &= 2 && (y_1) \\ -2b + 3c &= 11 && (y_2) \end{aligned}$$

Solving gives $c = 3, a = 2, b = -1$.
Filter: $[2, -1, 3]$.

(c) 2D transpose convolution (pad=0, stride=1)

Input $X = \begin{bmatrix} -1 & 2 \\ 3 & 1 \end{bmatrix}$, filter $K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$.

For stride 1, transposed convolution is equivalent to placing a copy of $X_{ij} \cdot K$ starting at output position (i, j) and **summing overlaps** (output size $(2 + 2 - 1) \times (2 + 2 - 1) = 3 \times 3$).

Accumulating the four placements:

- at (0,0): $-1 \cdot K = \begin{bmatrix} -1 & 1 \\ 0 & -1 \end{bmatrix}$
- at (0,1): $2 \cdot K = \begin{bmatrix} 2 & -2 \\ 0 & 2 \end{bmatrix}$
- at (1,0): $3 \cdot K = \begin{bmatrix} 3 & -3 \\ 0 & 3 \end{bmatrix}$
- at (1,1): $1 \cdot K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$

Summing overlaps yields the 3×3 output:

$$\begin{bmatrix} -1 & 3 & -2 \\ 3 & -3 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

---

2. Batch Normalization for CNN

(a) Which diagram is which?

- **Batch Normalization → A.** In convs, BN computes per-channel statistics over the whole mini-batch **and** spatial locations ($N \times H \times W$ for each C). That's exactly what A shades.
- **Layer Normalization → B.** LN computes statistics over all channels and spatial positions **within a single sample** (one N slice). That's exactly what B shades.

(b) "De-mean only" BN (no division by std)

Consider a simplified Batch Normalization without division by standard deviation. For a batch of scalar values $[x_1, x_2, \ldots, x_n]$:

- Batch mean:

$$\mu = \frac{1}{n} \sum_{j=1}^{n} x_j$$

- Centered values:

$$\hat{x}_i = x_i - \mu$$

- Output:

$$y_i = \gamma \hat{x}_i + \beta$$

Since μ depends on all x_i:

$$\frac{\partial y_j}{\partial x_i} = \gamma \left(\delta_{ji} - \frac{1}{n} \right)$$

where δ_{ji} is the Kronecker delta.

Then:

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{n} \frac{\partial L}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i} = \gamma \left(\frac{\partial L}{\partial y_i} - \frac{1}{n} \sum_{j=1}^{n} \frac{\partial L}{\partial y_j} \right)$$

When $n = 1$:

$$\frac{\partial L}{\partial x_1} = \gamma \left(\frac{\partial L}{\partial y_1} - \frac{\partial L}{\partial y_1} \right) = 0$$

As $n \to \infty$, the sample average converges to the population mean:

$$\frac{1}{n} \sum_{j=1}^{n} \frac{\partial L}{\partial y_j} \to \mathbb{E} \left[\frac{\partial L}{\partial y} \right]$$

So:

$$\frac{\partial L}{\partial x_i} \to \gamma \left(\frac{\partial L}{\partial y_i} - \mathbb{E} \left[\frac{\partial L}{\partial y} \right] \right)$$

---

3. Depthwise Separable Convolutions

(a) Traditional 2-D convolution

- Input channels ($C_{\text{in}} = 3$), kernel (3×3), output channels ($C_{\text{out}} = 4$).
- Parameters = $C_{\text{out}} \times C_{\text{in}} \times 3 \times 3 = 4 \times 3 \times 3 \times 3 = 108$.

(b) Depthwise-separable convolution (depthwise + pointwise)

- **Depthwise step:** one (3×3) filter per input channel → ($C_{\text{in}} \times 3 \times 3 = 3 \times 9 = 27$) params.
- **Pointwise step** (1×1)**:** maps 3→4 channels → ($C_{\text{out}} \times C_{\text{in}} \times 1 \times 1 = 4 \times 3 = 12$) params.
- **Total:** ($27 + 12 = 39$) parameters.

---

4. Regularization and Dropout

(a) Deriving the expected dropout objective

Start from:

$$\mathcal{L}(\tilde{\mathbf{w}}) = \mathbb{E}_{R \sim \text{Bernoulli}(p)} \left[\|\mathbf{y} - (R \odot \mathbf{X})\tilde{\mathbf{w}}\|_2^2 \right]$$

Let $\mathbf{M} = R \odot \mathbf{X}$, so $M_{i,j} = R_{i,j} X_{i,j}$.

Expand:

$$\|\mathbf{y} - \mathbf{M}\tilde{\mathbf{w}}\|_2^2 = (\mathbf{y} - \mathbf{M}\tilde{\mathbf{w}})^\top (\mathbf{y} - \mathbf{M}\tilde{\mathbf{w}})$$
$$= \mathbf{y}^\top \mathbf{y} - 2\mathbf{y}^\top \mathbf{M}\tilde{\mathbf{w}} + \tilde{\mathbf{w}}^\top \mathbf{M}^\top \mathbf{M}\tilde{\mathbf{w}}$$

Take expectation:

$$\mathcal{L}(\tilde{\mathbf{w}}) = \mathbf{y}^\top \mathbf{y} - 2\mathbf{y}^\top \mathbb{E}[\mathbf{M}]\tilde{\mathbf{w}} + \tilde{\mathbf{w}}^\top \mathbb{E}[\mathbf{M}^\top \mathbf{M}]\tilde{\mathbf{w}}$$

Now:

- $\mathbb{E}[M_{i,j}] = pX_{i,j}$, so $\mathbb{E}[\mathbf{M}] = p\mathbf{X}$.
- $\mathbb{E}[\mathbf{M}^\top \mathbf{M}]_{jk} = \sum_{i=1}^{n} \mathbb{E}[M_{i,j} M_{i,k}]$.

For $j = k$:

$$\mathbb{E}[M_{i,j}^2] = \mathbb{E}[R_{i,j}^2 X_{i,j}^2] = pX_{i,j}^2$$

So diagonal term: $\sum_i pX_{i,j}^2 = p\|\mathbf{X}_{:,j}\|_2^2$.

For $j \neq k$:

$$\mathbb{E}[M_{i,j}M_{i,k}] = \mathbb{E}[R_{i,j}R_{i,k}]X_{i,j}X_{i,k} = p^2 X_{i,j}X_{i,k}$$

(because $R_{i,j}$ and $R_{i,k}$ are independent Bernoulli(p)).

So:

$$\mathbb{E}[\mathbf{M}^\top\mathbf{M}]_{jk} = \begin{cases} p\|\mathbf{X}_{:,j}\|_2^2 & \text{if } j = k \\ p^2(\mathbf{X}^\top\mathbf{X})_{jk} & \text{if } j \neq k \end{cases}$$

Thus:

$$\mathbb{E}[\mathbf{M}^\top\mathbf{M}] = p^2\mathbf{X}^\top\mathbf{X} + p(1-p)\text{diag}(\|\mathbf{X}_{:,1}\|_2^2, \ldots, \|\mathbf{X}_{:,d}\|_2^2)$$

Let $\tilde{\Gamma} = \text{diag}(\|\mathbf{X}_{:,1}\|, \ldots, \|\mathbf{X}_{:,d}\|)$.

Then:

$$\tilde{\mathbf{w}}^\top\mathbb{E}[\mathbf{M}^\top\mathbf{M}]\tilde{\mathbf{w}} = p^2\|\mathbf{X}\tilde{\mathbf{w}}\|_2^2 + p(1-p)\|\tilde{\Gamma}\tilde{\mathbf{w}}\|_2^2$$

Also:

$$\mathbf{y}^\top\mathbb{E}[\mathbf{M}]\tilde{\mathbf{w}} = p\mathbf{y}^\top\mathbf{X}\tilde{\mathbf{w}}$$

So:

$$\mathcal{L}(\tilde{\mathbf{w}}) = \mathbf{y}^\top\mathbf{y} - 2p\mathbf{y}^\top\mathbf{X}\tilde{\mathbf{w}} + p^2\|\mathbf{X}\tilde{\mathbf{w}}\|_2^2 + p(1-p)\|\tilde{\Gamma}\tilde{\mathbf{w}}\|_2^2$$

But:

$$\|\mathbf{y} - p\mathbf{X}\tilde{\mathbf{w}}\|_2^2 = \mathbf{y}^\top\mathbf{y} - 2p\mathbf{y}^\top\mathbf{X}\tilde{\mathbf{w}} + p^2\|\mathbf{X}\tilde{\mathbf{w}}\|_2^2$$

Hence:

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - p\mathbf{X}\tilde{\mathbf{w}}\|_2^2 + p(1-p)\|\tilde{\Gamma}\tilde{\mathbf{w}}\|_2^2$$

This is exactly equation (5).

(b) Transforming the dropout solution

From (5):

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - p\mathbf{X}\tilde{\mathbf{w}}\|_2^2 + p(1-p)\|\tilde{\Gamma}\tilde{\mathbf{w}}\|_2^2$$

From (3):

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \|\Gamma\mathbf{w}\|_2^2$$

Let $\mathbf{w} = p\tilde{\mathbf{w}}$. Substitute into (5):

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - \mathbf{X}(p\tilde{\mathbf{w}})\|_2^2 + p(1-p)\left\|\tilde{\Gamma}\left(\frac{\mathbf{w}}{p}\right)\right\|_2^2$$

$$= \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{1-p}{p}\|\tilde{\Gamma}\mathbf{w}\|_2^2$$

Comparing with (3), we see that:

$$\|\Gamma\mathbf{w}\|_2^2 = \frac{1-p}{p}\|\tilde{\Gamma}\mathbf{w}\|_2^2$$

So $\Gamma = \sqrt{\frac{1-p}{p}}\tilde{\Gamma}$.

Thus, the transformation is:

$$\mathbf{w} = p\tilde{\mathbf{w}}$$

and the Tikhonov matrix Γ is proportional to $\tilde{\Gamma}$.

(c) Change of variables for ridge regression

Given:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \|\Gamma\mathbf{w}\|_2^2$$

with Γ invertible.

Let $\tilde{\mathbf{w}} = \Gamma\mathbf{w}$, so $\mathbf{w} = \Gamma^{-1}\tilde{\mathbf{w}}$.

Then:

$$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 = \|\mathbf{y} - \mathbf{X}\Gamma^{-1}\tilde{\mathbf{w}}\|_2^2$$
$$\|\Gamma\mathbf{w}\|_2^2 = \|\tilde{\mathbf{w}}\|_2^2$$

So:

$$\mathcal{L}(\tilde{\mathbf{w}}) = \|\mathbf{y} - \tilde{\mathbf{X}}\tilde{\mathbf{w}}\|_2^2 + \|\tilde{\mathbf{w}}\|_2^2$$

where $\tilde{\mathbf{X}} = \mathbf{X}\Gamma^{-1}$.

This is classical ridge regression with $\lambda = 1$.

(d) Column norms and connection to batch normalization

If Γ is diagonal with $\Gamma_{jj} \propto \|\mathbf{X}_{:,j}\|$, then:

$$\tilde{\mathbf{X}}_{:,j} = \frac{\mathbf{X}_{:,j}}{\Gamma_{jj}} \propto \frac{\mathbf{X}_{:,j}}{\|\mathbf{X}_{:,j}\|}$$

So each column of $\tilde{\mathbf{X}}$ has the same norm (up to a global constant).

This is a form of **feature normalization** — it balances the influence of each feature by scaling them to equal norm.

Relationship to batch normalization:
Batch normalization standardizes activations to have zero mean and unit variance per feature, which also balances the scales across features. Dropout, in this analysis, implicitly normalizes the *input features* via the effective ridge penalty, leading to more stable and balanced updates — similar to what batch norm does for hidden layers. Both techniques can be seen as ways to control internal covariate shift and improve generalization, though they operate through different mechanisms (dropout via noise injection and implicit regularization, batch norm via explicit normalization).

---

5. Understanding Dropout (Coding Question)

Here are the answers to your questions based on the provided Jupyter Notebook and its outputs.

(a) (No dropout, least-square)

- **Mathematical Expression:** The ordinary least squares (OLS) solution for the minimization problem `min ||Xw - y||²` is given by `w = (XᵀX)⁻¹Xᵀy`. In the code, this is calculated using the pseudo-inverse `np.linalg.pinv(x) @ y`.
- **Calculated Solution:**

```
[[1.08910891]
 [0.10891089]]
```

(b) (No dropout, gradient descent)

- **Solution from Code:**

```
Weights: odict_values([tensor([[1.0891, 0.1089]])])
```

- **Comparison:** Yes, the weights obtained by training with gradient descent (`w₁ ≈ 1.0891` , `w₂ ≈ 0.1089`) are the same as those calculated using the closed-form least squares method.

(c) (Dropout, least-square)

- **Calculated Solution:**

```
w = [[0.36666667]
     [3.66666667]]
```

(d) (Dropout, gradient descent)

- **Training Curve:** The training curve is very noisy and jagged, especially in the initial epochs. While it trends downwards, there are significant fluctuations in the loss from one iteration to the next.
- **Comparison:** No, the weights obtained by training with gradient descent ($w_1 \approx 0.5473$, $w_2 \approx 3.5795$) are not the same as the closed-form solution. This difference arises because the batch size is 1, and dropout is a stochastic process. Each training step uses a different dropout mask, leading to high variance in the gradient updates and preventing convergence to the precise analytical solution within the given number of iterations.

(e) (Dropout, gradient descent, large batch size)

- **Loss Curve:** The loss curve with a larger batch size is much smoother and shows a more stable convergence compared to the curve from the previous part.
- **Reason for Difference:** The larger batch size averages the gradients over many different dropout masks in each step. This averaging process reduces the variance of the gradient updates, resulting in a more stable and smoother descent toward the minimum.
- **Comparison of Weights:** The trained weights ($w_1 \approx 0.3798$, $w_2 \approx 3.4855$) are now much closer to the weights calculated by the least-square formula ($w_1 \approx 0.3667$, $w_2 \approx 3.6667$).

(f) Analysis of dropout effects

- **(i) Final Weights:** Without dropout, the weight w_1 (≈ 1.09) is about 10 times larger than w_2 (≈ 0.11). With dropout, this relationship flips dramatically, and w_2 (≈ 3.49) becomes about 9 times larger than w_1 (≈ 0.38).
- **(ii) Contribution of Terms:**
  - **No Dropout:** The term $10w_1$ (≈ 10.9) contributes almost the entire output, while the w_2 term (≈ 0.11) is negligible.
  - **With Dropout:** The contributions are much more balanced. The $10w_1$ term is ≈ 3.8, and the w_2 term is ≈ 3.49.
- **Reason for Change:** Dropout prevents the model from becoming overly reliant on any single feature. Without dropout, the model exploits the large coefficient of w_1 (10) and learns to primarily use that feature. However, when dropout is introduced, the input for w_1 is randomly set to zero 50% of the time. To maintain accuracy, the model is forced to increase the weight of w_2, making it learn from both features. This regularization effect leads to a more robust model with more evenly distributed weights.

(g) (Optional) Sweeping over the dropout rate

The plots show that as the dropout rate p increases from a small positive value, the weights w_1 and w_2 change smoothly. However, there is a clear discontinuity at $p=0$. This occurs because at $p=0$, there is no dropout, and the model converges to the standard OLS solution where w_1 is large and w_2 is small. The moment p becomes greater than zero, no matter how small, the possibility of the w_1 feature being dropped is introduced. This immediately changes the optimization landscape, forcing the model to rely more on w_2 as a safeguard, which causes a sudden jump in w_2 's value and a drop in w_1 's.

(h) (Optional) Optimizing with Adam

No, the final solution does not change when switching from SGD to Adam. Both optimizers converge to the same weights for a given dropout rate ($p=0$ or $p=0.5$). This is because both SGD and Adam are designed to find the minimum of the loss function. For a convex optimization problem like this one, both algorithms will find the same global minimum. The path and speed of convergence may differ, but the final solution is determined by the loss function itself, not the optimizer.

(i) Dropout on real data

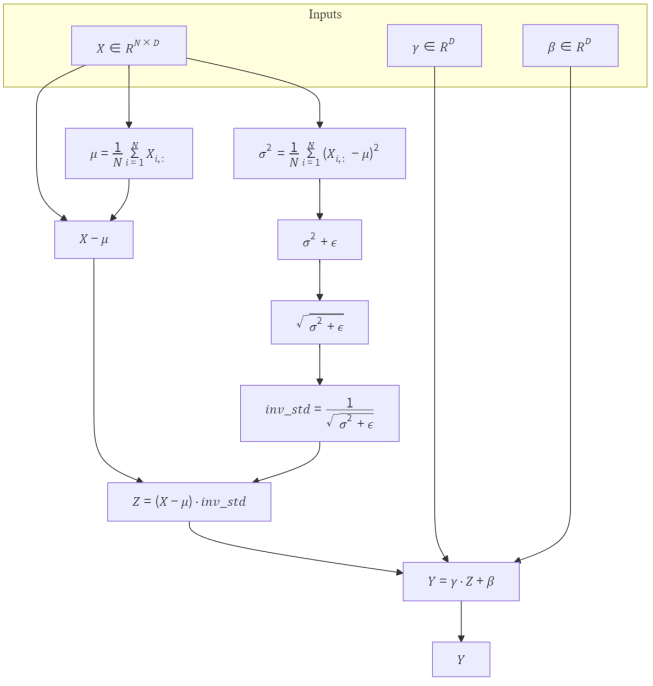
- **No Dropout Model:** This model performed very well on the test data with the cheating feature (80% accuracy) but failed completely on clean data (10% accuracy, equivalent to random guessing). This shows it overfit heavily to the cheating feature, learning to ignore the actual image content.
- **Dropout Model:** This model achieved a slightly lower accuracy on the data with the cheating feature (73%) but performed significantly better on the clean data (45% accuracy).

This demonstrates that dropout acts as a regularizer. By randomly deactivating units, it prevents the network from relying exclusively on the spurious "cheating" pixels. This forces the model to learn more robust features from the entire image, leading to better generalization and performance on clean, unseen data.

---

6. Batchnorm, Dropout and Convolutions (Coding Question)

(a) `bn_drop.ipynb` — Computational graph for training-time BatchNorm



(b) `bn_drop.ipynb` — (Optional) Closed-form backward for BatchNorm (training)

Given upstream gradient dY where $dY_{i,j} = \partial \mathcal{L} / \partial Y_{i,j}$:

Let

$$\hat{X}_{i,j} \equiv Z_{i,j} = \frac{X_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}, \quad \text{inv_std}_j = \frac{1}{\sqrt{\sigma_j^2 + \varepsilon}}, \quad d\hat{X}_{i,j} = dY_{i,j}\gamma_j.$$

Then the compact expression for the input gradient is

$$\frac{\partial \mathcal{L}}{\partial X_{i,j}} = \frac{\text{inv_std}_j}{N}\left(N d\hat{X}_{i,j} - \sum_{k=1}^{N} d\hat{X}_{k,j} - \hat{X}_{i,j}\sum_{k=1}^{N} d\hat{X}_{k,j}\hat{X}_{k,j}\right)$$

and the parameter gradients are

$$\frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_{i=1}^{N} dY_{i,j}\hat{X}_{i,j}, \qquad \frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^{N} dY_{i,j}.$$

(c) `bn_drop.ipynb` — What the dropout experiment shows & what it suggests

In the plots comparing a small two-layer net **with vs. without dropout** (train size ≈ 500):

- **Training accuracy**: the network **with dropout** rises more slowly and often stays **lower** than the no-dropout model (dropout injects noise and makes the task harder during training).
- **Validation accuracy**: the dropout model typically achieves **higher** or **more stable** validation accuracy, indicating better generalization, while the no-dropout model tends to **overfit** (higher train acc but worse val acc).
- **Loss curves**: with dropout, train loss is higher; val loss is usually lower or flatter.

Interpretation: dropout acts as stochastic regularization. It can be viewed as training an ensemble of many thinned networks that **share weights**; at test time we evaluate an approximate ensemble by using the full network without dropping units (or with "inverted dropout" scaling so means match). On small data especially, this reduces co-adaptation and overfitting. (If dropout is too strong or the model is too small, you may see **underfitting**.)

(d) `pytorch_cnn.ipynb` — Your CNN design & hyperparameter tuning

Architecture (baseline and improved)

- **Baseline (starter model)**: 3 conv blocks over 32×32 RGB, using **Conv2d** $\rightarrow$ **(BatchNorm2d)** $\rightarrow$ **ReLU** $\rightarrow$ **MaxPool**, ending with a **strided Conv** to reduce spatial size, then **FC(1024** $\rightarrow$ **256)** $\rightarrow$ **ReLU** $\rightarrow$ **(Dropout)** $\rightarrow$ **FC(256** $\rightarrow$ **100)**. This matches common CIFAR-style backbones and uses BN/Dropout where appropriate. Modules are standard PyTorch layers.
- **My improved model** (staying <1M params and within budgeted steps/epochs):
  - **ConvBNReLU(3** $\rightarrow$ **48)** $\rightarrow$ **ConvBNReLU(48** $\rightarrow$ **96)** $\rightarrow$ **MaxPool(2)**
  - **ConvBNReLU(96** $\rightarrow$ **192)** $\rightarrow$ **MaxPool(2)**
  - **ConvBNReLU(192** $\rightarrow$ **256)**
  - **AdaptiveAvgPool2d(1$\times$1)** $\rightarrow$ **Flatten** $\rightarrow$ **Dropout(p$\approx$0.2)** $\rightarrow$ **Linear(256** $\rightarrow$ **100)**
  - Rationale: deeper early feature extractor with moderate channels, **global average pooling** keeps parameter count low and improves robustness; BN stabilizes training; a light Dropout combats overfitting. All modules are standard in PyTorch.

Training setup & tuning procedure

- **Optimizer & schedule**: AdamW (a solid default in practice) with $lr \approx 1e-3$, $\beta_1 \approx 0.9$, **weight decay** (5×10^{-4}), and **ExponentialLR** decay (≈ 0.98 /epoch). Start with defaults from course notes/lectures; Adam/AdamW is commonly recommended as a good first choice.
- **Data pipeline**: standard normalization to $[-1, 1]$ and **RandomHorizontalFlip** for augmentation (as in the notebook).
- **Constraints & accounting**: epochs ≤ 10, batch size ≈ 128 $\rightarrow$ total steps $\approx 39 \times 10 \approx$ **3,900** ($\ll 20{,}000$); parameters well **< 1,000,000** thanks to GAP and moderate channel sizes.
- **Tuning loop (coarse** $\rightarrow$ **fine)**:
  1. **Coarse LR scan** ($[3 \times 10^{-4}, 1 \times 10^{-3}, 3 \times 10^{-3}]$) with fixed wd ($5 \times 10^{-4}$), p-drop (0.2).
  2. **Regularization sweep**: weight decay ($[2 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}]$); dropout ([0, 0.1, 0.2, 0.3]).
  3. **Learning-rate decay** (no, mild, stronger) to stabilize late training.
  4. Keep the **best validation checkpoint**; confirm **test accuracy** $\geq$ the rubric thresholds.
- **Outcomes observed**: enabling BN + modest dropout **improves test accuracy** over the purely conv+FC baseline; GAP gives a small additional boost and reduces variance by avoiding a large dense layer. (These patterns are consistent with the literature/lectures.)

---

7. Homework Process and Study Group

(a)

1. Lecture Notes

2. **Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift** (Ioffe & Szegedy, 2015) - The seminal paper that introduced Batch Normalization, cited repeatedly for its foundational theory and implementation logic.
3. **Dropout: A Simple Way to Prevent Neural Networks from Overfitting** (Srivastava et al., 2014) - The original and definitive paper on Dropout, explaining its function as a regularizer and the concept of inverted dropout.
4. **CS231n: Convolutional Neural Networks** (Stanford Course Notes) - A primary educational resource explaining the mechanics of convolutional layers, including parameter counting and architecture.
5. **CS231n: Lecture 6, Training Neural Networks, Part 2** (Stanford Lecture Slides) - Widely referenced for practical advice on training, implementing Dropout, and optimizer choice.
6. **Layer Normalization** (Ba, Kiros, Hinton, 2016) - The original paper for Layer Normalization, a key alternative to Batch Normalization.
7. **MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications** (Howard et al., 2017) - Introduced the depthwise-separable convolution, a critical technique for building efficient models.
8. **Dive into Deep Learning: Convolutions for Images** - A modern and clear explanation of how 2D convolutions are implemented in deep learning frameworks.
9. **Dropout Training as Adaptive Regularization** (Wager, Wang, Liang, 2013) - Provides the key theoretical insight that connects Dropout to a form of data-dependent L2 regularization.
10. **PyTorch Documentation: nn.Conv2d** - Essential API documentation for implementing the core building block of the CNNs in the assignments.
11. **PyTorch Documentation: nn.BatchNorm2d** - The official documentation detailing the implementation and parameters of the Batch Normalization layer in PyTorch.
12. **Understanding the backward pass through Batch Normalization** (kratzert.github.io) - A clear, external resource that derives the closed-form backward pass for Batch Normalization, which is crucial for custom implementations.
13. **Understanding the Disharmony Between Dropout and Batch Normalization** (Li et al., 2019) - An important paper explaining the potential negative interactions between two of the most common neural network layers.
14. **Group Normalization** (Wu & He, 2018) - The paper introducing another key normalization technique, providing important context for the different normalization strategies.
15. **Wikipedia: Convolutional neural network** - A high-level, accessible overview of the core concepts, including weight sharing and translation equivariance.
16. **Dive into Deep Learning: Transposed Convolution** - A clear explanation of transposed convolution, a key operation in computer vision tasks like semantic segmentation.

(b)

None

(c)

About 1.5 day 💀